

Ágora - Changelog Consolidado e Evolução do SaaS

Comparação entre a base antiga e a base atual do código, combinada com o histórico operacional do Lovable e as decisões estratégicas registradas ao longo do desenvolvimento.

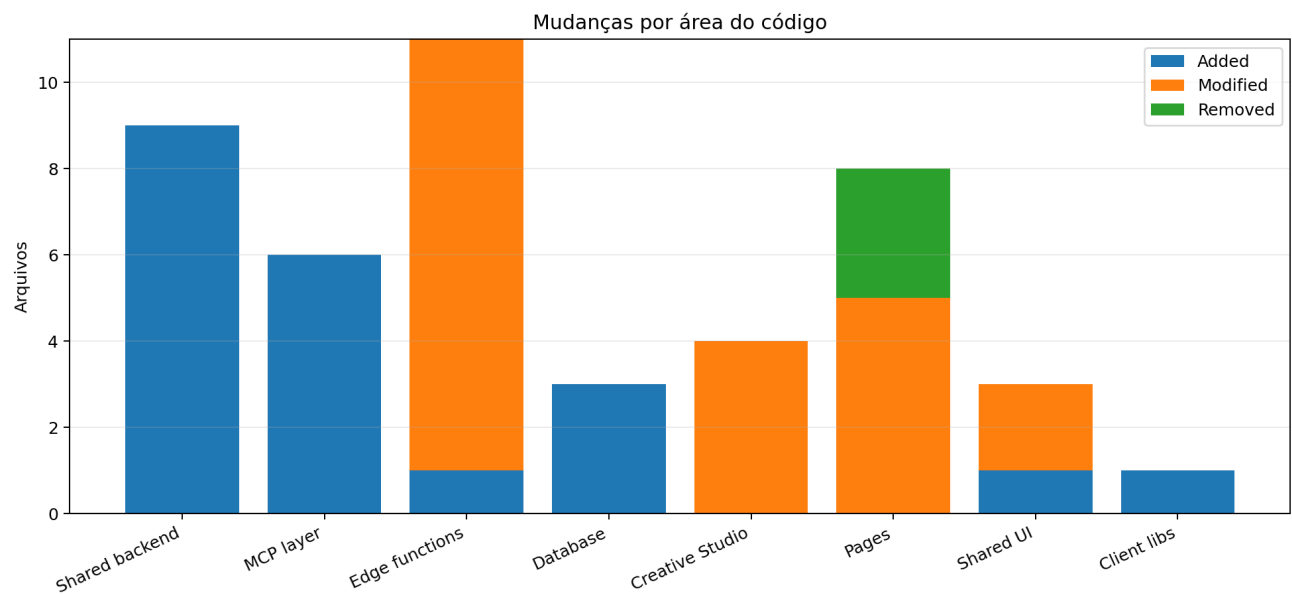
Arquivos totais	177 -> 198
Alterações centrais	+24 / -3 / 26 modificados
Pages app	15 -> 12
Migrations	13 -> 16

Leitura executiva

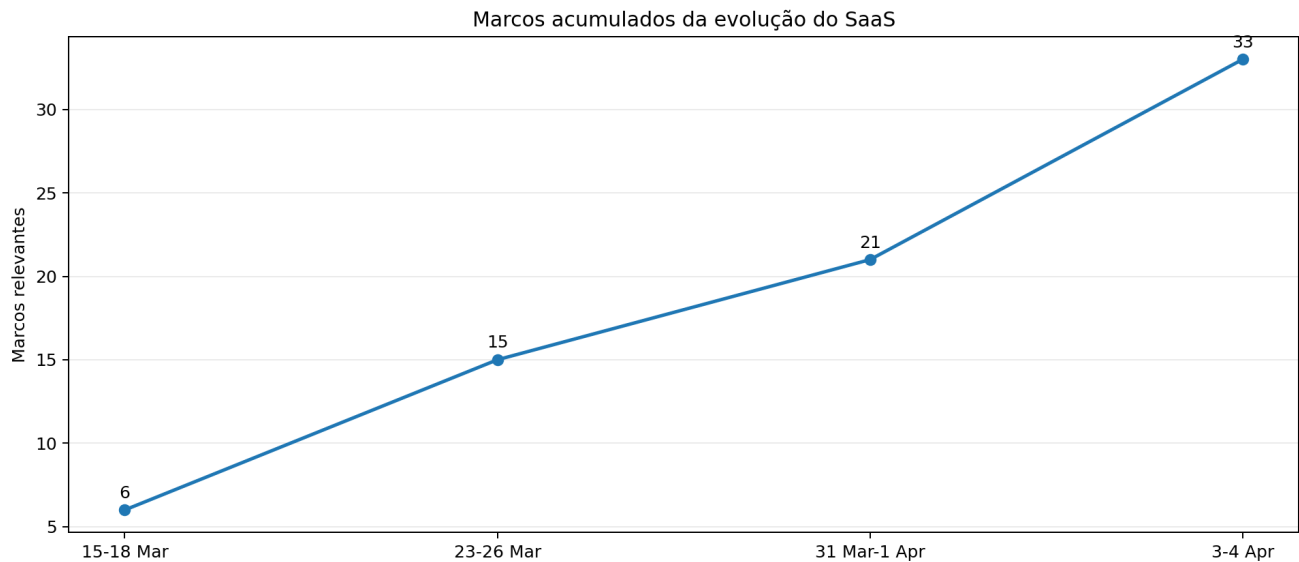
- O código atual não representa apenas uma soma de novas features. Ele mostra uma mudança de maturidade: menos páginas órfãs, mais carregamento sob demanda, menos duplicação no frontend e muito mais estrutura compartilhada no backend.
- A principal mudança sistêmica foi a passagem de um SaaS funcional, porém mais acoplado, para uma base que já começa a operar por camadas: rotas, chat compartilhado, backend modular, kernel multiagente, camada MCP-ready, subsistema criativo e comparador.
- Ao mesmo tempo, o histórico revela alguns pontos de drift entre documentação e código real. O PDF destaca esses casos para que o changelog funcione não só como vitrine, mas também como instrumento de governança técnica.

1. Evidências quantitativas da evolução

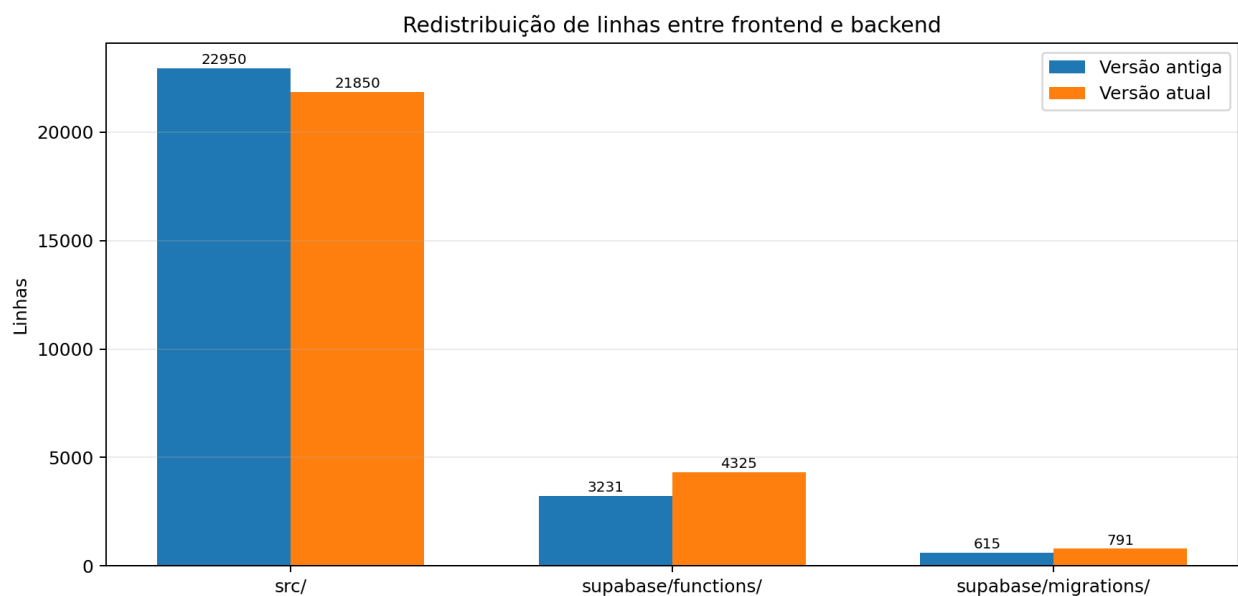
Mudanças por área do código



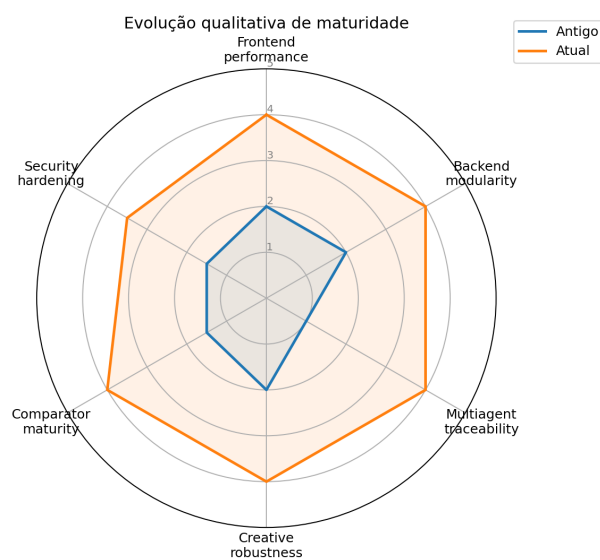
Marcos acumulados da evolução do SaaS



Redistribuição de linhas entre frontend e backend



Evolução qualitativa de maturidade (escala 1-5)



2. Escala de evolução usada neste relatório

Nível	Significado
N1	Ajuste pontual
N2	Melhoria funcional
N3	Consolidação de fluxo
N4	Novo subsistema
N5	Evolução arquitetural

3. Linha do tempo dos marcos principais

Janela	Qtd. de marcos	Resumo
15-18 Mar	6	Persistência de chat, histórico, dashboard, UX básica
23-26 Mar	9	Creative Studio, artboards, jobs, migração Gemini, README
31 Mar-1 Apr	6	Comparador, dashboards, mobile, retry/fallback criativo
3-4 Apr	12	Lazy loading, helpers compartilhados, kernel, MCP, segurança

4. Changelog consolidado por domínio

Persistência de chat e histórico [N4] - Fluxo do usuário

Antes o contexto do intake se perdia com F5 e o histórico não aparecia de forma utilizável. Agora cada conversa cria ID próprio, salva mensagens em banco, restaura via ?c=, exibe abas de Conversas/Análises e usa o conteúdo da primeira mensagem como título.

Dashboard com conversas recentes e evolução [N3] - Fluxo do usuário

O dashboard passou a mostrar Conversas Recentes e um gráfico de evolução das notas, reduzindo a distância entre análise concluída, retomada de contexto e acompanhamento do progresso.

Botões de ação contextual na Nova Análise [N2] - UX/Entrada

Os antigos atalhos foram substituídos por modos ativáveis como Gerar imagem, Pesquisa de mercado e Gerar campanha, com adaptação específica para mobile.

Anexos multimodais no chat [N4] - Entrada multimodal

Arquivos de texto passam a ser lidos como conteúdo textual e PDFs/imagens/binários seguem pela API multimodal do Gemini, inclusive quando o envio ocorre sem texto digitado.

Context Cards interativos [N4] - Coleta de contexto

O pedido de contexto deixou de depender só de texto corrido. O sistema ganhou cards com opções, perguntas abertas, fila de perguntas e envio único ao final.

Context Cards endurecidos [N3] - Coleta de contexto

Mensagens antigas deixaram de permanecer clicáveis; o comparador e os chats principais passaram a respeitar a regra de último assistant + não streaming.

Chat com Estrategista contextualizado [N3] - Relatório/UX

O antigo botão foi trocado por um painel lateral deslizante, persistido em banco, que já abre com o contexto completo da análise.

Scroll e navegação do chat estabilizados [N3] - Relatório/UX

Foram removidos saltos de scroll, criado o botão Mais recente e corrigidos problemas de remount e stale closure envolvendo query params e estado das mensagens.

Creative editor inline reformulado [N4] - Criativo

O editor inline ganhou comportamento mais próximo de Canva: duplo clique para texto, toolbar flutuante, persistência como mensagem e fluxo bloqueado durante geração.

Creative Studio em workspace de artboards [N5] - Criativo

O estúdio deixou de ser um canvas isolado e evoluiu para um workspace com artboards, grid infinito, mini mapa, persistência, auto-save e editor focado.

Vínculo creative_job -> artboard [N5] - Criativo

Jobs criativos passaram a abrir diretamente o artboard correto, inclusive para chats sem analysis_id; isso exigiu creative_jobs compatível com análise opcional e artboards persistidos em banco.

Robustez de geração de imagem [N5] - Criativo/Backend

generate-image e generate-creative ganharam retry com backoff, fallback de modelo, feedback de falha parcial, upload para storage e HTML com gradient fallback quando a imagem falha.

Layout engine e UX do editor [N3] - Criativo/Frontend

Texto fora de margem, rotação, sliders, cores, minimap, clique/duplo clique, teclado e autosave foram lapidados para reduzir perda de estado e conflitos entre edição e reabertura.

Comparador de Campanhas lançado [N5] - Comparador

Foi criado um módulo próprio com edge function dedicada, página de chat própria, suporte a imagens/arquivos, renderização markdown dedicada e link na sidebar.

Dashboard estilo Artifacts [N4] - Comparador

O comparador passou a parsear um bloco [DASHBOARD] estruturado e renderizar KPI cards, radar, barras e banner da campanha vencedora, reduzindo a parede de texto.

Regra first-party vs third-party [N2] - Comparador

A seção de recomendação executiva passou a ser exibida apenas para campanhas próprias, omitida quando todas são third-party.

Responsividade e mobile do comparador [N3] - Comparador

O comparador ganhou tela cheia no mobile, header próprio, padding/larguras corrigidos e fechamento automático do menu lateral.

Saneamento estrutural de rotas [N4] - Arquitetura frontend

Rotas órfãs e imports mortos foram removidos; AssetsPage, CampaignOptimizerPage e HistoryPage saíram do runtime, e o botão Novo chat ficou consistente entre sidebar e dashboard.

Lazy loading nas páginas pesadas [N4] - Arquitetura frontend

App.tsx passou a usar React.lazy + Suspense nas páginas de maior peso, adiando a separação entre bundle inicial e telas como Estúdio Criativo e Comparador.

Camada compartilhada de chat [N4] - Arquitetura frontend

Tipos e helpers duplicados foram centralizados em chatHelpers.ts e ChatMessageBubble.tsx, consolidando bolhas, auto-resize, scroll e limpeza de conteúdo.

Backend com helpers compartilhados [N5] - Arquitetura backend

Foi criada a pasta _shared com módulos para CORS, erros, Gemini, IBGE, geração de imagem, Supabase, validação e rate limit. Isso removeu duplicação e preservou contratos.

Kernel multiagente incremental [N5] - Arquitetura backend

A análise passou a registrar runs e run_steps, com 5 etapas rastreáveis, duração, modelo, tokens e bloco Pipeline de Análise no relatório.

Camada MCP-ready [N5] - Arquitetura backend

O projeto ganhou catálogo tipado de tools/resources/prompts, reality layer reaproveitável e benchmark resource injetado em analyze-campaign.

Segurança e resiliência [N5] - Segurança

Entraram validação com Zod nas funções críticas, rate limiting por RPC e endurecimento de escrita em daily_usage_counters e rate_limit_log.

Rate limiting validado em produção lógica [N3] - Segurança

Os três chats principais foram testados com rate limiting ativo e continuaram retornando 200 com SSE coerente.

Documentação arquitetural atualizada [N2] - Governança

README e memórias técnicas foram atualizados para refletir Fabric.js 6, auto-save, novas edge functions, kernel e MCP layer - embora o código ainda mostre alguns pontos de drift.

5. Mudanças estruturais confirmadas no diff de código

- Arquivos adicionados: 24 (com destaque para `_shared` backend, camada MCP e novos artefatos de memória técnica).
- Arquivos removidos: 3 (`AssetsPage.tsx`, `CampaignOptimizerPage.tsx` e `HistoryPage.tsx` saíram do runtime atual).
- Arquivos modificados com maior impacto: `analyze-campaign`, `optimize-campaign`, `ReportChatBlock`, `AnalysisChatPage`, `ComparatorDashboard`, `NewAnalysisPage`, `generate-creative` e `generate-image`.
- A pasta `src/` encolheu 1100 linhas, enquanto `supabase/functions/` cresceu 1094 linhas - um indício claro de extração de complexidade do frontend para o backend compartilhado.
- O banco ganhou 3 migrations novas, com ênfase em `analysis_runs`, `run_steps`, endurecimento de `daily_usage_counters` e `rate_limit_log/check_rate_limit()`.

Arquivos novos mais significativos

Arquivo	Papel resumido
<code>src/components/ChatMessageBubble.ts</code>	Bolha unificada de mensagens do chat
<code>src/lib/chatHelpers.ts</code>	Helpers compartilhados de mensagem/scroll/resize
<code>supabase/functions/_shared/gemini.ts</code>	Cliente/helper comum para chamadas Gemini
<code>supabase/functions/_shared/kernel.ts</code>	Kernel incremental para runs e steps
<code>supabase/functions/_shared/mcp/*</code>	Catálogo MCP-ready de tools/resources/prompts
<code>supabase/functions/_shared/rate-limit.ts</code>	Aplicação de rate limit por RPC
<code>supabase/functions/_shared/validation.ts</code>	Validação Zod compartilhada
<code>supabase/migrations/*04025018*.sql</code>	<code>analysis_runs</code> e <code>run_steps</code>
<code>supabase/migrations/*04051957*.sql</code>	<code>rate_limit_log</code> e <code>check_rate_limit()</code>

6. Observações de consistência e lacunas

- O código atual confirma a maior parte da evolução descrita no histórico, mas há pelo menos um ponto de drift: `analyze-campaign` ainda mantém `gemini-2.5-pro` como fallback, apesar de o README/histórico terem anunciado sua remoção.
- O comparador foi endurecido em várias passadas. O histórico registra uma tentativa de fixar sempre `gemini-2.5-flash` para evitar JSON malformado; porém o snapshot atual ainda preserva roteamento para `gemini-2.5-flash-lite` em alguns cenários de 3+ campanhas/conversas longas.
- A evolução mais nítida não está no volume bruto de linhas do frontend. O `src/` encolheu, enquanto a camada `supabase/functions` cresceu e ficou mais modular, sugerindo migração de complexidade para o backend compartilhado.

Leitura interpretativa

A versão atual está claramente mais madura do que a antiga, mas o ganho não é linear. O frontend ficou mais limpo e previsível, enquanto o backend se tornou o principal lugar de complexidade. Isso é positivo, desde que a governança acompanhe: contratos congelados, evals, observabilidade e revisão contínua do que já divergiu entre README, prompts e snapshot real.

7. Apêndice - Python para gerar os gráficos

O arquivo completo acompanha esta entrega como `agora_changelog_charts.py`. Abaixo está um recorte do script usado para gerar os gráficos incluídos no PDF.

```
import json
import math
import matplotlib.pyplot as plt

with open("agora_changelog_dataset.json", "r", encoding="utf-8") as f:
    data = json.load(f)

# 1) Mudanças por área
areas = list(data["changes_by_area"].keys())
added = [data["changes_by_area"][a]["added"] for a in areas]
modified = [data["changes_by_area"][a]["modified"] for a in areas]
removed = [data["changes_by_area"][a]["removed"] for a in areas]
fig, ax = plt.subplots(figsize=(11, 5.2))
x = range(len(areas))
ax.bar(x, added, label="Added")
ax.bar(x, modified, bottom=added, label="Modified")
ax.bar(x, removed, bottom=[a + m for a, m in zip(added, modified)], label="Removed")
ax.set_title("Mudanças por área do código")
ax.set_ylabel("Arquivos")
ax.set_xticks(list(x))
ax.set_xticklabels(areas, rotation=25, ha="right")
ax.legend()
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig("chart_changes_by_area.png", dpi=180)
plt.close(fig)

# 2) Timeline
labels = [w[0] for w in data["waves"]]
counts = [w[1] for w in data["waves"]]
cum = []
acc = 0
for c in counts:
    acc += c
    cum.append(acc)
fig, ax = plt.subplots(figsize=(10.8, 4.8))
ax.plot(labels, cum, marker="o", linewidth=2)
for i, v in enumerate(cum):
    ax.annotate(str(v), (i, v), textcoords="offset points", xytext=(0, 7), ha="center")
ax.set_title("Marcos acumulados da evolução do SaaS")
ax.set_ylabel("Marcos relevantes")
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig("chart_timeline.png", dpi=180)
plt.close(fig)

# 3) Redistribuição de LOC
cats = list(data["loc_compare"].keys())
old_vals = [data["loc_compare"][c][0] for c in cats]
new_vals = [data["loc_compare"][c][1] for c in cats]
fig, ax = plt.subplots(figsize=(9.8, 4.8))
width = 0.35
idx = range(len(cats))
ax.bar([i - width/2 for i in idx], old_vals, width=width, label="Versão antiga")
ax.bar([i + width/2 for i in idx], new_vals, width=width, label="Versão atual")
ax.set_title("Redistribuição de linhas entre frontend e backend")
ax.set_ylabel("Linhas")
ax.set_xticks(list(idx))
ax.set_xticklabels(cats)
ax.legend()
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig("chart_loc_shift.png", dpi=180)
plt.close(fig)

# 4) Radar qualitativo
labels = list(data["maturity_dims"].keys())
old_m = [data["maturity_dims"][l][0] for l in labels]
new_m = [data["maturity_dims"][l][1] for l in labels]
angles = [n / float(len(labels)) * 2 * math.pi for n in range(len(labels))]
```

Arquivos complementares entregues junto com este PDF:

- `agora_changelog_completo.md` - versão em texto puro, mais fácil de copiar e adaptar;
- `agora_changelog_charts.py` - script Python para regenerar os gráficos;
- `agora_changelog_dataset.json` - base de dados usada pelos gráficos;